Centro Universitário de Excelência
Sistemas de Informação

MecâniQA: persistindo os dados com JDBC e MySQL

Autores: Hélio Nunes Moraes Junior
Kevilyn Bitencourt da Cruz
Marcelo Henrique Fernandes Mauricio
Murillo Rigaud dos Santos Oliveira
Rodrigo Alves Oliveira

MecâniQA AUTOMOTIVE TECH

Vitória da Conquista, 2026

Agenda

MecâniQA
AUTOMOTIVE TECH

Objetivo: mostrar como a MecâniQA passou a guardar seus dados no MySQL com JDBC puro, mantendo o código organizado em camadas e padronizado.

1. Contexto:
Da memória para o MySQL

2. Arquitetura:
Camadas e diagrama de classes

3. Acesso a dados:
Script SQL, JDBC e Template Method

4. Dados e erros:
ResultSet Mapper e exceções customizadas

5. Resultados:
Requisitos da OAT 3 entregues

Da memória para o MySQL

- **Persistência:** guardar os dados fora da memória do programa, num lugar que sobrevive ao reinício.
- **JDBC puro:** todo SQL escrito à mão; JPA, Hibernate e Spring Data são proibidos.
- **Mesmo contrato:** quem usa a API continua com os mesmos endpoints e o mesmo JSON.

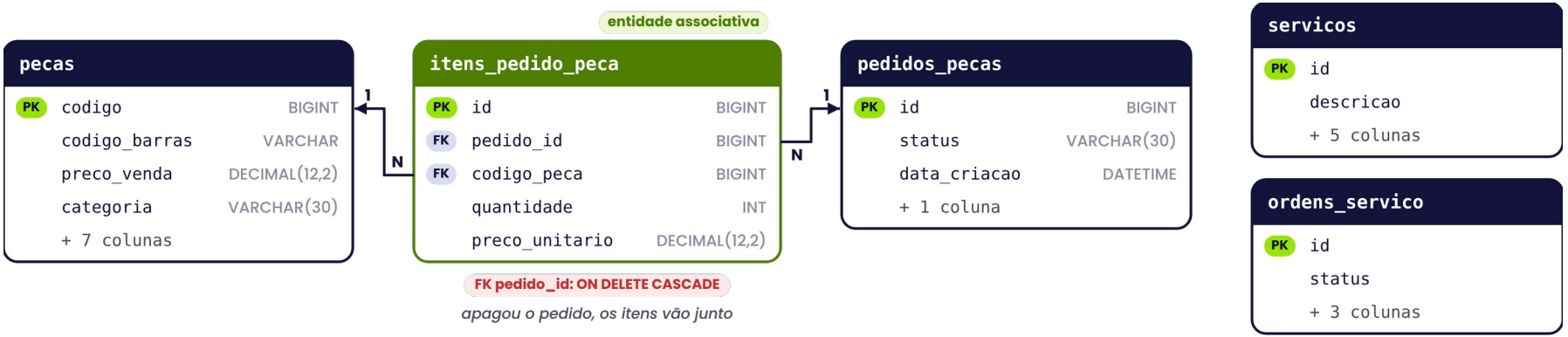


Arquitetura em camadas

- **Separação de responsabilidades:** cada camada tem um único papel e só conversa com a vizinha.
- **Baixo acoplamento:** trocar a forma de guardar os dados não muda o Controller.
- **Camada de Service:** nova na OAT 3, concentra toda a regra de negócio.



Do objeto à tabela



JDBC

API do Java para falar com o banco: a Connection abre, o PreparedStatement executa e o ResultSet traz as linhas.

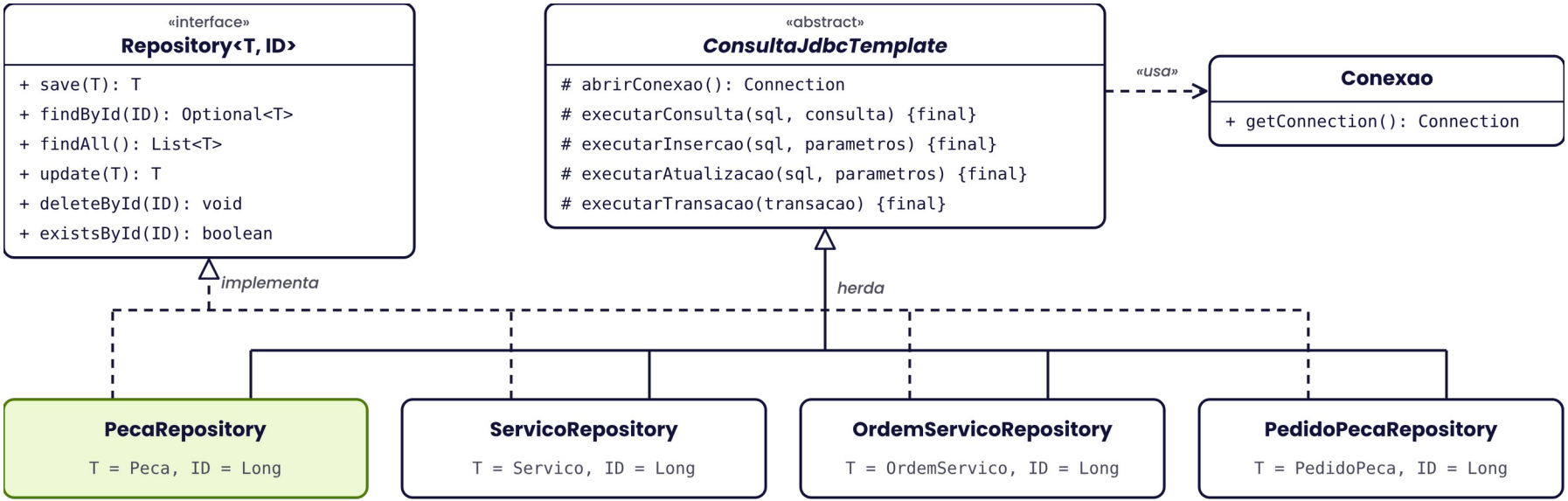
Script versionado

O script_mecaniqa.sql cria o banco e as tabelas (DDL) e fica no repositório junto com o código.

Credenciais fora do código

Usuário e senha ficam no arquivo .env; no Git vai apenas o .env.example.

Diagrama de classes



Interface genérica

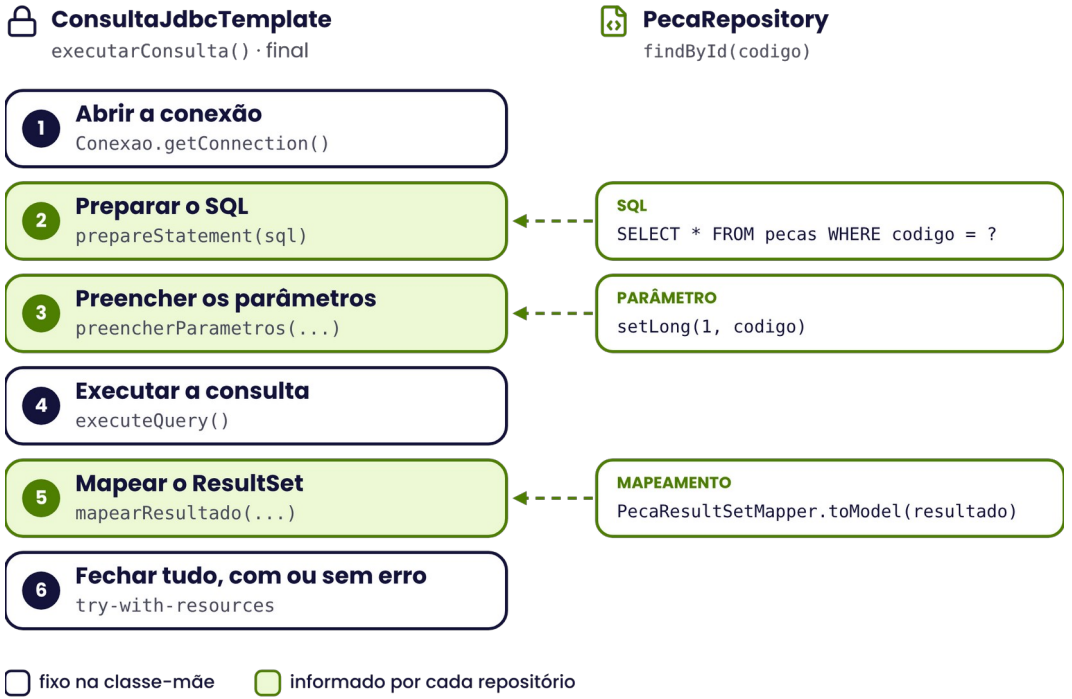
Um único contrato de CRUD; T e ID são trocados pelo tipo de cada entidade.

Herança

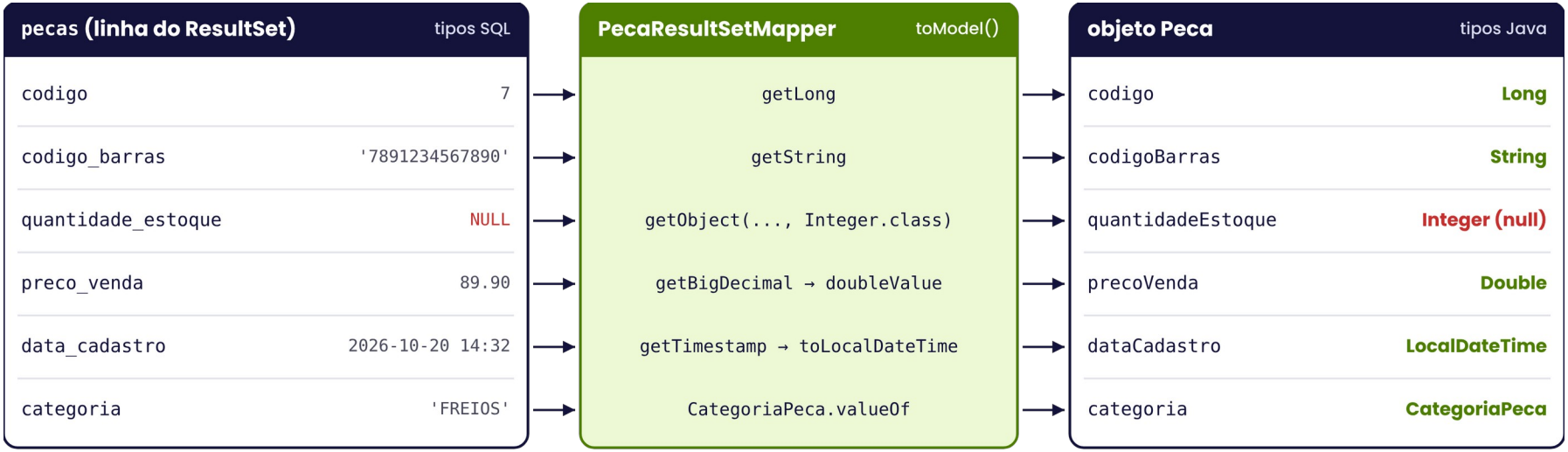
Os quatro repositórios estendem ConsultaJdbcTemplate e reaproveitam o acesso ao banco.

Template Method

- **Padrão comportamental:** a classe-mãe define o passo a passo; as filhas preenchem só o que muda.
- **final:** os métodos do template não podem ser sobrescritos, então o esqueleto nunca se perde.
- **Sem repetição:** abrir, executar e fechar a conexão está escrito uma única vez.



ResultSet Mapper



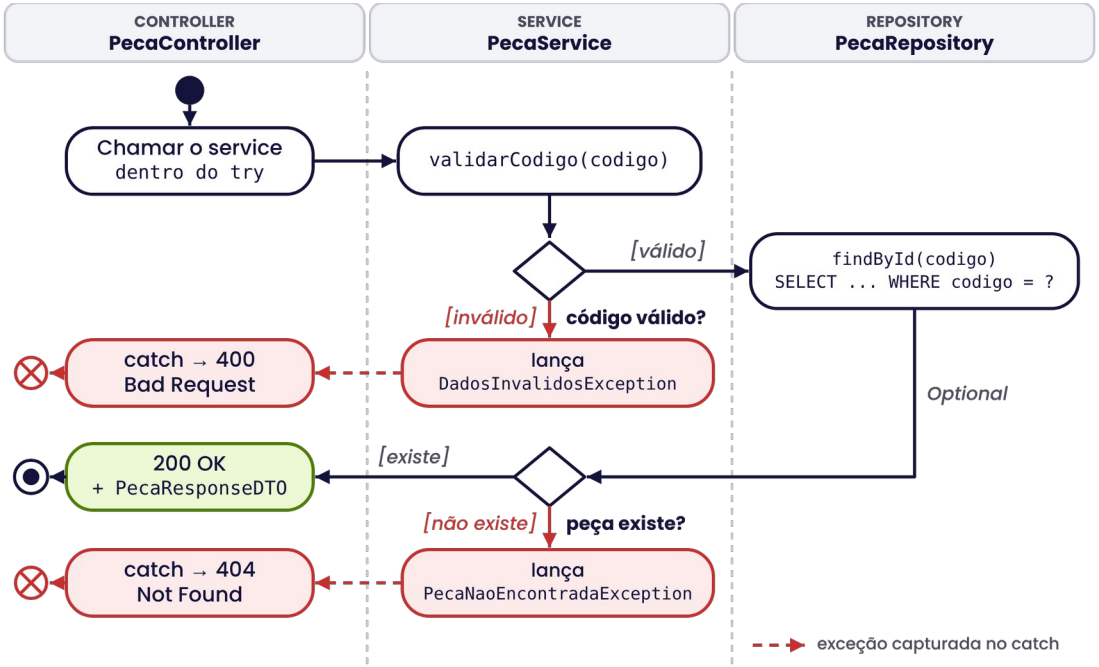
ResultSet
O resultado de um SELECT, percorrido linha a linha com next().

ResultSet Mapper
Lê cada coluna, converte o tipo do SQL para o do Java e trata os nulos.

Reuso da OAT 2
O mapper da Ordem de Serviço monta o objeto usando o Builder.

Exceções customizadas

- **Exceção customizada:** classe própria que dá nome a um erro do negócio.
- **Unchecked:** estendem RuntimeException, então não exigem throws em cada método.
- **Sem @ExceptionHandler:** cada endpoint trata seus erros com try/catch explícito.



Considerações finais

Requisitos da OAT 3

- MySQL com JDBC, sem ORM
- Script SQL versionado
- Repository<T, ID> em todos os repositórios
- Template Method no acesso a dados
- ResultSet Mappers dedicados
- Exceções customizadas com try/catch
- Camada de Service entre Controller e Repository

Para quem usa a API: os mesmos endpoints e o mesmo JSON da OAT 2, agora com dados que continuam lá depois que o servidor reinicia.

MecâniQA AUTOMOTIVE TECH

Referências

- MECÂNIQA. OAT 3 – Persistindo os meus Dados. 2026.
- EQUIPE PORTO ALEGRE. Repositório do projeto, 2026. github.com/Meizolas/mecaniQA-api-porto-alegre
- GAMMA, E. et al. Padrões de Projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.
- FOWLER, M. Padrões de Arquitetura de Aplicações Corporativas. Porto Alegre: Bookman, 2006.
- ORACLE. JDBC Basics – The Java Tutorials. docs.oracle.com/javase/tutorial/jdbc
- ORACLE. MySQL 8.0 Reference Manual. dev.mysql.com/doc/refman/8.0/en